

Gate Calibration: A Fidelity Management Framework for Persistent AI Systems

How to Measure What Compression Destroys

Borja Odriozola Schick

April 2026

Abstract

Persistent AI systems compress accumulated context to survive within finite processing windows, but current practice treats compression as a token-reduction problem—optimizing for volume rather than meaning. We show that compression fidelity decomposes into four independent dimensions (factual accuracy, structural coherence, trajectory preservation, and tonal/strategic nuance) and that managing these dimensions requires calibrated external gating rather than self-supervised summarization. We establish that fidelity duration—the range over which output quality remains acceptable—is independent of context window size, governed instead by the model’s degradation rate. This reframes scaling debates from capacity to calibration: the binding constraint is not how much a system can hold but how long it can think well. We present a calibration loop in which human review functions as measurement infrastructure, generating the dimensional data that tunes the system’s own compression thresholds. Empirical validation across a 25-iteration persistent engineering build confirms that external calibration quality directly modulates effective degradation.

1 Introduction

Anthropic—the laboratory behind Claude and among the most technically sophisticated organizations working on large language models—publishes a recommended approach to context management for long-running AI workflows. Their Agent SDK offers automatic context compaction: when token usage exceeds a configurable threshold, the system injects a summary prompt, the model summarizes its own conversation, the history is cleared, and processing resumes with only the summary [1]. The approach is well-engineered. On a modular customer-service workflow, it achieves 58.6% token reduction across two compaction events, processing five independent tickets within a single agentic run.

This is the current state of practice. It is also, we argue, a solution to the wrong problem.

Context compaction manages token count. It does not manage fidelity. The distinction is not semantic. Compaction optimizes for factual preservation as its primary fidelity criterion—whether key details survive the summary. It applies a scalar gate—a token-count threshold that triggers compression mechanically, without reference to what the compression will destroy along other dimensions. It provides no external verification—the model summarizes itself and

trusts the result, the precise self-referential structure that Odriozola-Schick [3] proved leads to incomplete self-verification. And it operates within a single run—when the session ends, everything is gone. No persistence, no accumulation, no learning across boundaries.

None of this is a criticism of the engineering. Anthropic identified the right operational pressure—linear token growth in agentic loops—and solved an important subset of the problem. Their compaction works well for workflows composed of independent processing units where factual retention is the primary fidelity requirement. The five-ticket customer-service demonstration is a good use case precisely because tickets are independent: compressing Ticket 2’s resolution before processing Ticket 3 loses nothing that Ticket 3 needs.

But most persistent AI systems are not processing independent tickets. They are maintaining strategic context across sessions, tracking how understanding developed over time, preserving the weight and nuance of accumulated decisions. For these systems, the question is not how many tokens survived compression but how much meaning survived—and along which dimensions meaning was lost. Compaction provides no framework for asking this question, let alone answering it.

This paper provides that framework.

The argument follows a three-paper arc. The Root Theorem [2] established the diagnosis: any bounded lossy channel exhibits monotonic quality degradation with accumulated volume, and persistent systems necessarily converge on homeostatic memory architectures—accumulate, compress, rewrite, shed. Odriozola-Schick [3] established the prognosis: the system performing the compression cannot verify the fidelity of its own output, because the verification instrument operates within the same degraded channel it attempts to evaluate. This paper provides the treatment: a calibrated external gating framework that decomposes fidelity into four measurable dimensions, establishes gate positions derived from degradation rates rather than capacity limits, and implements a calibration loop in which human review simultaneously validates the current compression and trains the system’s fidelity thresholds.

The central contribution is the four-dimensional fidelity decomposition. Current practice treats compression quality as a single axis—accurate or not, good enough or not. We show that compression fidelity decomposes into four independent dimensions, each measurable, each with different sensitivity to degradation, and each weighted differently by different use cases. This decomposition is the conceptual tool that explains both when existing approaches like compaction work (independent-unit workflows requiring factual retention only) and when they are destructive (trajectory-dependent workflows where structural coherence, temporal ordering, or strategic nuance carries irreplaceable signal).

The paper formalizes the degradation-fidelity relationship and its independence from window size (§2), introduces the four fidelity dimensions (§3), presents the calibration protocol and provenance-aware compression framework (§§4–5), derives the two-pass compression architecture (§6), validates the framework empirically (§7), and positions it against existing work before discussing implications (§§8–10).

2 Formal Setup

2.1 Three Parameters

Every persistent AI system that compresses its context operates under three parameters:

Context window size W . The hard constraint imposed by model and access tier. Currently ranging from 128K to 1M tokens across production deployments.

Degradation rate D . The rate at which output quality decreases as context fills, expressed as fractional fidelity loss per 100K tokens. D is not a single number—it decomposes along the four fidelity dimensions introduced in Section 3 and varies by model. For current frontier models, aggregate D is estimated at 1–2% per 100K tokens. D must be estimated initially, then calibrated empirically per deployment through the protocol described in Section 4. We write D_{eff} for the degradation rate as observed in a specific deployment, which may differ from the model’s baseline D due to calibration quality, content composition, and information provenance.

Target fidelity F_{target} . The minimum acceptable compression quality for a given deployment. This is a design decision, not a model property—the parameter the system architect sets based on what the use case requires.

2.2 The Fidelity Function

At position P tokens consumed, with degradation rate D :

$$F(P) = 1 - \frac{D \cdot P}{100,000} \quad (1)$$

This is a linear first approximation. Real degradation profiles exhibit sublinear behavior in early context (the model tolerates moderate fill with minimal quality loss) and superlinear behavior near capacity (the cliff effect observed in practice). We adopt the linear model for tractability and note that calibration corrects for non-linearity empirically—the framework’s power lies in the calibration loop, not in the precision of the initial model.

The gate fires where fidelity drops below target:

$$P_{\text{gate}} = \frac{100,000 \times (1 - F_{\text{target}})}{D} \quad (2)$$

2.3 The Independence Result

P_{gate} is independent of W .

This is the critical insight most practitioners miss. A larger context window lets the system carry more tokens, but it does not extend the range over which output quality remains acceptable. Window size governs *capacity*—how much the system can hold. Degradation rate governs *fidelity duration*—how long the system can think well. These are independent constraints.

Consider a concrete example. At $D = 2\%$ per 100K tokens with $F_{\text{target}} = 97\%$ (a strategic intelligence profile), the gate position is:

$$P_{\text{gate}} = \frac{100,000 \times 0.03}{0.02} = 150,000 \text{ tokens} \quad (3)$$

This gate fires at 150K tokens regardless of whether the window is 200K or 1M. On a 200K window, the gate fires at 75% capacity—the system compresses before it fills. On a 1M window, the gate fires at 15% capacity—the system compresses having used barely a sixth of its available space. In both cases, the fidelity constraint binds before the capacity constraint.

The practical consequence is immediate: most serious deployments are fidelity-constrained, not capacity-constrained. Scaling the window does not solve the quality problem. Calibrating the degradation rate does.

2.4 Gate Position Tables

The following tables show gate positions for two degradation rate estimates across five fidelity profiles.

Table 1: Gate positions at $D = 2\%$ per 100K tokens (conservative estimate).

F_{target}	Profile	P_{gate}	% of 200K	% of 1M
98%	Strategic (tight)	100K	50%	10%
97%	Strategic (relaxed)	150K	75%	15%
95%	Professional	250K	125%*	25%
93%	Task execution	350K	175%*	35%
90%	High-volume	500K	250%*	50%

Table 2: Gate positions at $D = 1\%$ per 100K tokens (optimistic estimate).

F_{target}	Profile	P_{gate}	% of 200K	% of 1M
98%	Strategic (tight)	200K	100%	20%
97%	Strategic (relaxed)	300K	150%*	30%
95%	Professional	500K	250%*	50%
93%	Task execution	700K	350%*	70%
90%	High-volume	1,000K	500%*	100%

*Gate exceeds window. System is capacity-constrained, not fidelity-constrained. In these cases, the zone system (§2.5) applies relative to W rather than P_{gate} , and the window boundary functions as the effective gate.

The tables reveal a pattern: at conservative degradation estimates, even a 1M-token window is fidelity-constrained for strategic applications. At optimistic estimates, the 1M window becomes capacity-constrained only for high-volume, low-depth work. The degradation rate, not the window, is the binding variable.

2.5 Zone System

For any deployment, the gate position P_{gate} induces a zone structure governing operational behavior:

Zone ratios are derived from operational requirements: yellow provides approximately 5–8 turns of landing room and red approximately 3 turns, assuming ~ 800 tokens per turn average.

Table 3: Operational zone system derived from gate position.

Zone	Range	Behavior
Green	0 to $0.64 \times P_{\text{gate}}$	Full fidelity. Work freely.
Yellow	0.64 to $0.88 \times P_{\text{gate}}$	Start landing. Finish current thread.
Red	0.88 to $1.0 \times P_{\text{gate}}$	Close within 3 turns.
Gate	$> P_{\text{gate}}$	Close immediately.

These ratios are defaults calibrated for current frontier models; the calibration loop (Section 4) adjusts them empirically.

The zone system converts the abstract fidelity function into operational protocol. A system architect does not need to monitor $F(P)$ continuously—the zones translate the mathematics into behavioral rules that any deployment can follow.

3 Fidelity Is Multi-Dimensional

3.1 The Problem with Aggregate Fidelity

Current approaches to compression quality—including Anthropic’s context compaction—treat fidelity as a single axis. A summary either preserves the important information or it does not. This framing is adequate for workflows where only one kind of information matters, but it collapses distinctions that are load-bearing for persistent systems.

Consider a persistent system that has maintained context across thirty sessions with a user. The accumulated memory includes: project milestones and deadlines (facts), the causal chain by which early architectural decisions constrained later ones (structure), the order in which the user’s understanding of their own problem evolved (trajectory), and the distinction between the user’s settled convictions and their tentative explorations (nuance). A compression that preserves all the facts but flattens the causal chain has high factual accuracy and low structural coherence. A compression that preserves both facts and structure but scrambles the temporal ordering has high accuracy, high coherence, and low trajectory preservation. These are different failure modes, not different magnitudes of the same failure.

3.2 Four Dimensions

We decompose compression fidelity into four independently measurable dimensions:

Factual accuracy. Did compression preserve what actually happened? Decisions, dates, names, commitments, technical specifications, numerical results. This is the dimension that existing approaches target—the primary fidelity criterion in token-reduction frameworks such as Anthropic’s compaction. It is necessary but not sufficient.

Structural coherence. Do relationships between ideas survive compression? Causation, dependency, contradiction, hierarchy. A compressed memory that lists decisions without recording which decisions constrained which others has preserved the facts but lost the structure. Structural coherence measures whether the compressed representation supports the same inferences as the original.

Trajectory preservation. Does temporal sequence survive? The order in which understanding developed, the path from early confusion to later clarity, the sequence of attempts and failures that led to the current approach. Trajectory carries information that no static snapshot can recover—the fact that Approach B was adopted after Approach A failed is different from the fact that Approach B was adopted. Losing trajectory means losing the reasoning that justifies current state.

Tonal/strategic nuance. Does compression preserve the *weight* of things? The distinction between a settled decision and a tentative proposal. Between a major strategic commitment and a passing observation. Between a user’s deep conviction and their casual speculation. Nuance is the most fragile dimension and the hardest to measure. It is also, for strategic applications, the most valuable. A system that treats every statement as equally weighted cannot distinguish signals from noise in accumulated context.

3.3 Dimensional Independence

These dimensions are not reducible to each other. A compression can score perfectly on factual accuracy while destroying structural coherence (list of facts, no relationships). It can preserve structure while scrambling trajectory (correct dependency graph, wrong temporal ordering). It can maintain facts, structure, and trajectory while flattening nuance (everything preserved, but a major decision and a minor observation carry identical weight).

This independence has an immediate engineering consequence: aggregate fidelity scores mask dimensional failure. A compression that scores 95% aggregate might be 100% on factual accuracy, 98% on structural coherence, 95% on trajectory preservation, and 87% on nuance—and that 87% might be the dimension that matters most for the use case. One-dimensional measurement conceals the binding constraint. Existing model evaluation benchmarks can be reinterpreted as partial measurements along individual fidelity dimensions—a connection we develop in subsequent work on dimensional measurement.

3.4 Use-Case Fidelity Profiles

Different use cases weight the four dimensions differently, producing distinct fidelity profiles. We identify four canonical profiles:

Strategic intelligence (e.g., a coordination instance maintaining long-range context across a research program). All four dimensions are critical. Small errors compound through repeated compression cycles. Factual errors corrupt the record. Structural errors break the reasoning chain. Trajectory errors obscure why current state is justified. Nuance errors flatten the distinction between settled understanding and open questions. F_{target} : 97–98% across all dimensions.

Professional/client-facing (e.g., a legal or financial assistant maintaining client context). Factual accuracy is critical—compliance depends on it. Structural coherence is high—the logic connecting facts must survive. Trajectory and nuance are moderate—the client’s history matters but the precise order of every discovery does not. F_{target} : 95–97%, with factual accuracy as the binding dimension.

Task execution (e.g., a coding assistant maintaining project context across development sessions). Factual accuracy is critical for active tasks—file names, function signatures, bug

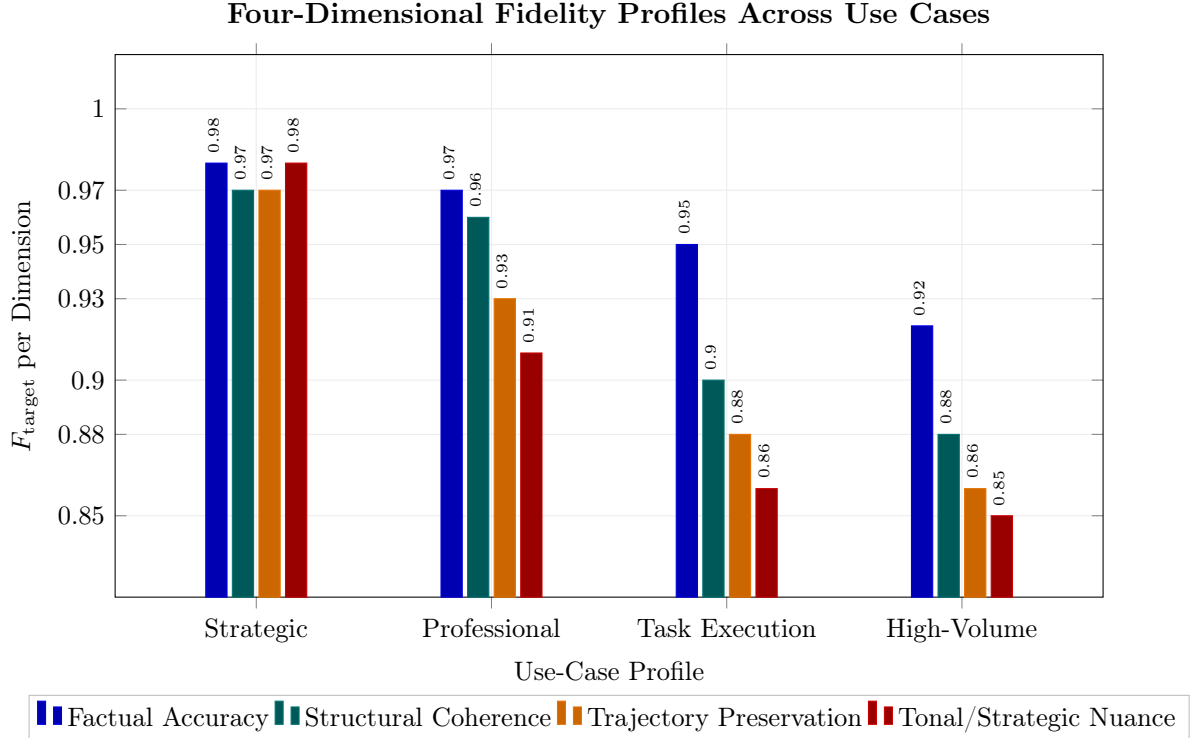


Figure 1: Four-dimensional fidelity target profiles across four canonical use cases. The strategic intelligence profile demands near-uniform high fidelity across all four dimensions. Professional/client-facing applications weight factual accuracy highest with progressively lower requirements for structure, trajectory, and nuance. Task execution concentrates fidelity budget on factual accuracy. High-volume applications accept lower thresholds across all dimensions, with cost as the binding constraint. The same formal framework produces different gate behaviors for each profile: dimensional weighting determines which dimension binds first and therefore where the gate fires.

descriptions must be precise. Other dimensions are moderate to low—the system needs to know what to build, not the full history of how the architecture was decided. F_{target} : 93–95%, with critical content protected through structural placement outside the compression cycle.

High-volume, low-depth (e.g., a customer service system handling independent queries with light personalization). Factual accuracy is moderate—the system needs approximate context, not perfect recall. All other dimensions are low. Cost is the binding constraint, not fidelity. F_{target} : 90–93%.

These profiles are not arbitrary categories. They are derived from the dimensional decomposition—each profile is a weighting vector across four axes. The same formal apparatus that produces strategic gates at 150K tokens produces high-volume gates at 500K tokens, from the same degradation rate, by adjusting which dimensions bind.

3.5 Dimensional Gate Binding

When fidelity is multi-dimensional, the gate fires when *any* dimension drops below its target. The binding dimension—the one that triggers the gate first—depends on the model’s degradation profile across dimensions and the use case’s weighting.

We hypothesize that nuance degrades fastest and factual accuracy last, but systematic dimensional measurement is needed to confirm any ordering—this is the subject of the forward measurement agenda (Section 9). If the hypothesis holds, it would explain a phenomenon practitioners have observed informally: the same model that handles coding tasks well at high context loads produces subtly wrong strategic advice at the same load. The aggregate degradation would be identical; the dimensional profile would not. The framework generates this as a testable prediction: use cases where nuance binds should gate significantly earlier than use cases where factual accuracy binds, even at identical aggregate degradation rates.

4 The Calibration Loop

4.1 The Human as Measurement Infrastructure

The Gödel result [3] establishes that no system can verify the fidelity of its own compression from within the compressed channel. The calibration loop provides the engineering response: an external agent—a human—who observes the compression output, rates its fidelity along four dimensions, and feeds those ratings back into the system’s gate parameters.

This is not oversight. Oversight checks output after the fact—did the system do the right thing? Calibration shapes the channel before and during operation—how much context can the system carry before quality degrades, and along which dimensions does it degrade first? The human in the calibration loop is functioning as a measurement instrument: reading the compressed output, comparing it against their understanding of the original, and generating dimensional ratings that the system cannot produce for itself.

The distinction matters because it reframes the human’s role from gatekeeper to sensor. A gatekeeper approves or rejects. A sensor generates data that tunes the system. The calibration loop does both simultaneously—the human’s approval decision validates the current compression, and the dimensional ratings from that approval train the system’s understanding of when and how quality degrades.

4.2 Protocol

Initial deployment. Set D conservatively (2% per 100K tokens). This produces early gates—the system compresses sooner than it needs to. Conservative initialization is deliberate: it is cheaper to discover that gates can move outward (losing some conversational headroom in early sessions) than to discover that gates should have fired earlier (losing irreplaceable context).

Calibration phase (sessions 1–10). After each compression event, the human rates the compressed output against the four fidelity dimensions. The system tracks: token position at compression, fidelity rating per dimension, session composition (ratio of interpretive to reference content), and provenance composition (ratio of recoverable to ephemeral content, defined in Section 5).

Each calibration session generates a data point: at position P , under these conditions, the human assessed fidelity at these dimensional ratings. Across 5–10 sessions, the system accumulates enough data to estimate D per dimension with usable precision.

Calibrated vs. Uncalibrated Fidelity Over Sessions

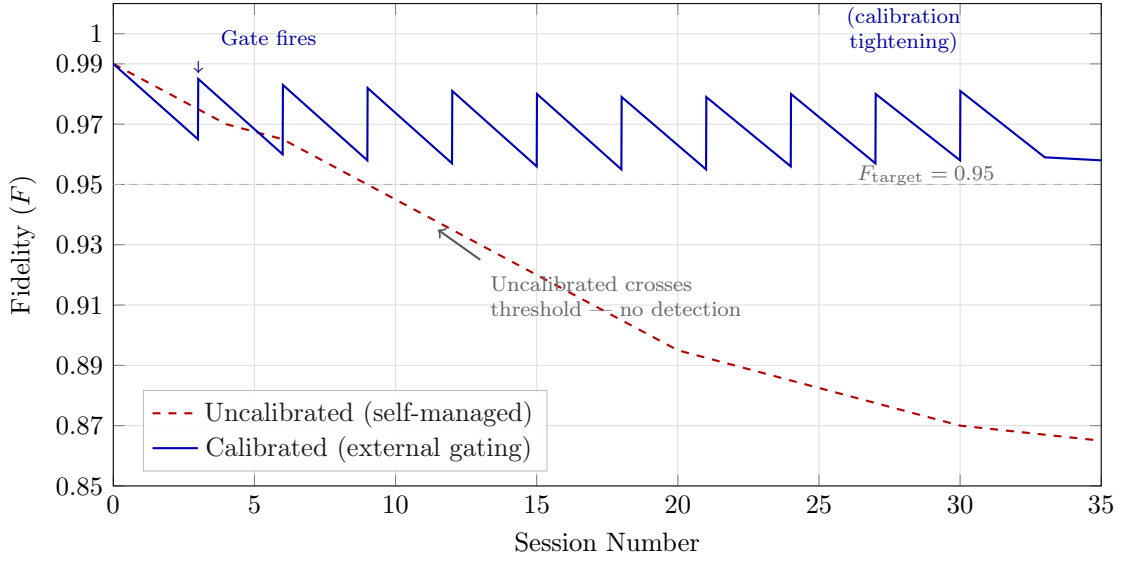


Figure 2: Fidelity divergence between calibrated and uncalibrated persistent systems over 35 sessions. The calibrated system (solid line) exhibits bounded sawtooth oscillation: fidelity drops during accumulation phases and recovers at each external gate event. The sawtooth amplitude narrows over time as the calibration flywheel tightens. The uncalibrated system (dashed line) drifts monotonically downward, crossing the $F_{\text{target}} = 0.95$ threshold near session 10 with no recovery mechanism. The divergence grows with each cycle: the cost of omitting external calibration compounds. Data anchored to empirical observations from a persistent context architecture operating across 110+ sessions and 22+ compression cycles.

Adjustment. If quality remains high beyond conservative gate positions, D decreases and gates shift outward. If quality degrades before the gate fires, D increases and gates shift inward. With four-dimensional tracking, adjustment becomes dimension-specific—the system learns not just that quality dropped but which dimension dropped and by how much.

Steady state. D stabilizes per deployment profile. Gates are calibrated. The calibration loop continues—each compression event generates fresh data—but adjustments become marginal. The system has learned its own degradation characteristics through external measurement.

4.3 The Calibration Flywheel

The calibration loop is self-improving. The human reviewing compression quality simultaneously performs two functions: validating the current compression (is this good enough?) and generating the data that calibrates future compressions (at what position, under what conditions, did quality degrade along which dimensions?). Each review makes the next compression more precisely gated.

This compounding is not metaphorical. Empirical evidence from a 25-iteration persistent build demonstrates the mechanism concretely (Section 7): as the human’s understanding of the system’s failure modes sharpened across iterations, the specificity of calibration input increased, and effective degradation decreased measurably. The calibration process calibrates itself.

4.4 Why Another Model Cannot Replace the Human

A natural objection: if the problem is self-verification, why not use a second model as the verification agent? This relocates the self-referential constraint but does not resolve it. The verifier model operates in its own bounded channel, subject to its own $D > 0$. It cannot verify its own verification fidelity for the same structural reasons the primary model cannot verify its compression fidelity. The Gödel result applies to any bounded channel, not to a specific model.

The pattern is already visible in current practice. Memory architectures that consolidate session records autonomously—promoting provisional notes to verified knowledge, merging session summaries into long-term memory without external review—are performing self-referential verification whether they use one model or two. A second model summarizing the first model’s output is a longer pipeline, not an escape from the bounded-channel constraint. Each model in the chain operates within its own context window, subject to its own degradation, and unable to certify the fidelity of its own contribution. The regress terminates only when an agent outside any bounded channel—the human—provides the verification.

This does not mean that model-based assistance in the calibration loop has no value. A second model can flag potential issues, highlight discrepancies, and reduce the human’s cognitive load. But the verification decision—the judgment that compression quality along a given dimension meets the required threshold—requires an agent outside the channel. The human is that agent. This is an architectural requirement, not a temporary limitation pending better automation.

5 Provenance-Aware Compression

5.1 Not All Information Requires Equal Preservation

Compression under a fidelity budget faces a resource allocation problem: which information receives the limited preservation effort? Generic compression treats all content equally—every fact, every inference, every reference consumes the same fidelity budget. This is wasteful because different pieces of information have fundamentally different recovery characteristics.

We introduce a provenance classification that distinguishes information by its recoverability:

Recoverable facts originate from systems of record—databases, knowledge bases, APIs, version control systems. If compression drops them, they can be re-injected from source at full fidelity in the next session. Preserving them through compression is redundant: the fidelity budget spent on them is wasted.

Ephemeral-verified facts exist only in the conversation. They were generated during interaction, confirmed by the human, and have no external source of truth. If compression drops them, they are permanently lost. The compressed memory is the only persistence layer for this information.

Unverified inferences are the system’s own conclusions—patterns it identified, predictions it made, connections it drew. These may be correct or may be hallucinated. Their provenance is internal, and their verification status is unknown.

5.2 The Three-Bin Framework

Provenance classification produces three compression bins with distinct fidelity allocations:

Table 4: Provenance-based compression bins and fidelity allocation.

Bin	Compression Instruction	Rationale
Recoverable	Strip content, preserve pointer	Re-injected from source at full fidelity.
Ephemeral-verified	Full fidelity budget	Irreplaceable signal. If lost, lost forever.
Unverified inference	Stage for human gate	Cannot self-verify. Hold pending review.

The provenance distinction is architectural, not inferential. The system architect declares source provenance in the system’s configuration—which data sources are systems of record, which content is conversation-derived. At compression time, the compressor receives provenance instructions and allocates fidelity budget accordingly. No machine-learning inference is needed; the classification is a design decision encoded in the architecture.

5.3 The Staged Inference Buffer

The third bin—unverified inference—operationalizes the Gödel result at the compression boundary. Inferences generated by the system during a session are held in a pending section of the compressed output. At the compression approval point, these inferences are surfaced to the human as discrete accept/reject decisions.

Accepted inferences are promoted to the ephemeral-verified bin—they become part of the persistent record at full fidelity. Rejected inferences are dropped. The system cannot verify its own conclusions, so it stages them for external verification rather than promoting them silently.

The staged inference buffer is deliberately minimal. It does not ask the human to evaluate the system’s entire output—only the pieces whose provenance is internal and whose verification status is unresolved. This bounds the human’s cognitive load while ensuring that the most epistemically uncertain content receives external review.

5.4 The Provenance Flywheel

Provenance-aware compression creates a compounding efficiency loop. When the compressor knows which facts are recoverable, it concentrates the full fidelity budget on irreplaceable signal. This raises compression quality at equivalent gate positions. Higher quality means lower effective degradation. Lower effective degradation means more headroom per session. More headroom means richer conversations. Richer conversations produce more understanding worth preserving.

The improvement is architectural, not parametric. It does not depend on a better model or a larger window—it depends on the system knowing the structure of its own information landscape. A system that compresses blindly wastes budget on recoverable facts. A system that compresses with provenance awareness concentrates its limited fidelity budget where loss is permanent.

6 Two-Pass Compression Architecture

6.1 The Position-Fidelity Relationship

The fidelity function $F(P)$ implies that compression quality depends on where in the context window the compression occurs. A compression performed at $P = 150,000$ tokens operates under already-degraded fidelity. A compression performed at $P = 10,000$ tokens operates near-losslessly. This relationship creates an architectural opportunity: if the system can separate its compression into two passes at different positions, the highest-stakes pass can run in the cleanest channel.

6.2 The Architecture

Session compression (Pass 1). At the end of a working session, the system compresses the session’s accumulated context into a compact record. This compression runs at the session’s current position P_{session} , which is by definition in the yellow or red zone (the zone system triggers the compression). Fidelity is $F(P_{\text{session}})$ —good, because the gate enforces minimum quality, but not perfect.

Memory rewrite (Pass 2). In a fresh context—a new API call with only the system’s memory file as input—the system reprocesses the session record and integrates it into persistent memory. This compression runs at position $P \approx 10,000$ tokens (the memory file plus the session record). At this position, $F(P)$ is near-lossless—the degradation contribution from a 10K-token context is negligible at any plausible D .

The two-pass architecture decouples session compression from memory integration. The first pass tolerates slightly lower fidelity because the second pass reprocesses it from a near-perfect channel. The highest-stakes operation—integrating new understanding into the persistent record that shapes all future sessions—always runs in the cleanest available channel.

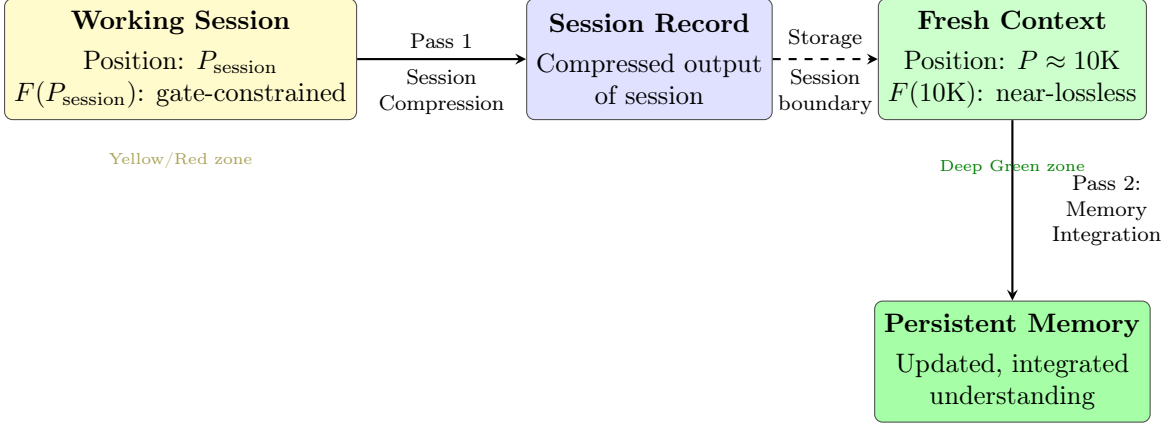
6.3 The Compounding Path

The architectural claim is straightforward: the second pass contributes negligible additional degradation because it operates at a position where $F(P)$ is near unity. The session compression pass is the binding constraint on per-cycle fidelity, and the gate system ensures that pass never drops below F_{target} .

Whether degradation compounds across many cycles—and if so, at what rate—is an open empirical question. The two-pass architecture bounds per-cycle degradation but does not resolve the longitudinal dynamics. Preliminary observation suggests that the memory rewrite’s near-lossless channel provides a stabilizing effect, but systematic measurement across dozens of compression cycles with dimensional tracking is needed to characterize the compounding behavior. This is part of the forward research agenda (Section 9).

6.4 Multi-Model Extension

The two-pass architecture naturally supports model routing. The session pass—which prioritizes responsiveness and domain competence—can run on one model. The memory rewrite—which



The second pass contributes negligible degradation. The session compression pass is the binding constraint on per-cycle fidelity. The highest-stakes operation runs in the cleanest channel.

Figure 3: Two-pass compression architecture. Pass 1 compresses the working session at position P_{session} (gate-constrained, yellow/red zone fidelity). The session record crosses the session boundary into storage. Pass 2 loads only the memory file and session record into a fresh context at $P \approx 10K$ tokens (near-lossless, deep green zone), where the highest-stakes operation—integrating new understanding into persistent memory—runs in the cleanest available channel. The architectural separation ensures that per-cycle fidelity is bounded by the gate system (Pass 1) while memory integration contributes negligible additional degradation (Pass 2).

prioritizes nuance preservation and structural coherence—can run on a different model optimized for those dimensional profiles. Model selection at each pass is a derived consequence of the four-dimensional fidelity framework: each pass has a dimensional priority, and models have dimensional strengths. The 25-iteration empirical build used distinct model tiers for session work versus coordination, and their differing fidelity profiles informed the routing decision.

We defer the full treatment of model-specific degradation profiles and routing strategies to subsequent work on degradation fine structure. The architectural point here is that two-pass compression creates the separation of concerns that makes dimensional model routing possible.

7 Empirical Evidence

7.1 The Dataset

We present evidence from a 25-iteration persistent engineering build—a real deployment in which a human-gated compression system was used to develop a functioning persistence engine. The system operated across multiple sessions with different models, maintaining continuity through the calibration architecture described above. The full dataset contains seventeen findings across multiple task categories; this paper presents the four that validate the calibration framework directly. The complete task-type decomposition is the subject of subsequent work.

7.2 Finding 1: The Scoping Gate

The same work product—identical code changes to the same files—lands at 14% or 28% context consumption depending on whether the human provides the complete solution in the calibration

input or describes the problem and lets the system investigate.

This finding has three instances in the dataset. In Iteration 15, a cross-cutting scope that would normally require the system to read across multiple interfaces and reason about their relationships was collapsed to a pattern-following task by providing exact text and exact insertion locations. Context consumption dropped from the 22–31% range characteristic of cross-cutting work to 14%. Iteration 19 extended this to the extreme: when the human provided exact text at an exact location, the system consumed 3% of context with 269 reasoning tokens—all dataset minimums. Iteration 20b demonstrated the same pattern in the opposite direction: restructuring a broadly scoped task into a bounded one reduced consumption from 28% to 16%.

The operational implication is direct: the human at the compression boundary is not merely approving or rejecting—the human is a measurement instrument whose calibration quality directly modulates system performance. The difference between 14% and 28% context consumption on the same task is not model capability. It is calibration precision.

7.3 Finding 2: Membrane Failure Produces Mode Shift

In Iteration 17, the calibration input—the handoff document specifying the task—was absent from the workspace. The system did not degrade. It did not produce lower-quality code. It changed what it produced entirely: instead of executing code changes, it generated an architectural analysis of what changes would be needed and why.

This is not graceful degradation. This is mode shift. Without external calibration, the system does not get worse at the intended task—it becomes a different system performing a different task. The prediction model, calibrated for code execution, predicted 24–28% consumption. The actual consumption was 12%—but 12% spent on analysis rather than execution.

The finding validates the Gödel result operationally: the system cannot detect its own mode shift. From the system’s perspective, it performed useful work—and it did. The architectural analysis was competent. But it was not the specified work. The self-referential constraint means the system has no mechanism to distinguish between “executing the task as specified” and “producing something useful given available context.” The external calibration input is not context for the task—it is the signal that selects which kind of work the system performs.

7.4 Finding 3: The Transcription Floor

When external calibration is maximized—exact instructions, exact code, exact location—context consumption drops to 3% with 269 reasoning tokens. All dataset minimums. At this operating point, effective degradation approaches zero because the system is functioning as a transcription instrument rather than a reasoning agent.

This finding establishes the dynamic range of the calibration lever. The same system, on comparable tasks, varies from 3% to 44% context consumption across the dataset. The difference is not model capability, task difficulty, or context window size. It is the precision of the external calibration input. D_{eff} is not a fixed property of the model. It is a jointly determined variable: the model’s baseline degradation rate modulated by the quality of external calibration.

7.5 Finding 4: Bidirectional Calibration

Specifying the required structure of the system’s return signal—approximately 40 tokens of format specification in the system prompt—produces 100% structured compliance in the system’s output. Without this specification (Iteration 21), the system produced a one-line summary. With it (Iteration 22), the system produced a seven-section structured report with verification evidence, line numbers, and confidence scores.

This finding closes the calibration loop. The human calibrates the input channel (specifying the task with precision, as Finding 1 demonstrates). The human also calibrates the output channel (specifying what the return signal must contain). Without output calibration, the system under-reports, and the calibration loop cannot function because the human needs structured data from the system to make informed approval decisions.

The operational cost is minimal: approximately 40 tokens of format specification produces a qualitative transformation in the system’s reporting—the human calibrating the instrument that measures the instrument.

7.6 Synthesis

The four findings converge on a single conclusion: the human in the calibration loop is measurement infrastructure. Finding 1 shows that calibration precision directly modulates execution cost. Finding 2 shows that absent calibration produces mode shifts invisible to the system. Finding 3 shows that the calibration lever’s dynamic range spans two orders of magnitude. Finding 4 shows that the calibration loop requires bidirectional instrumentation—the human must calibrate both what the system receives and what the system reports.

Together, they validate the gate calibration framework as operational rather than theoretical. The framework does not describe what should happen in principle—it describes what was measured across 25 iterations of a real engineering build.

8 Related Work

Context compaction, as demonstrated by Anthropic’s Agent SDK [1], manages the token-count problem for single-run agentic workflows. It optimizes for token reduction with factual retention as the implicit fidelity criterion—a one-dimensional approach to what this paper shows is a four-dimensional problem. Compaction is well-suited to independent-unit workflows—batch processing, ticket handling, sequential tasks with no cross-task dependencies—where factual accuracy is the dominant fidelity requirement. Gate Calibration extends to trajectory-dependent workflows where structural coherence, temporal ordering, or strategic nuance carries irreplaceable signal. The gap between compaction and calibration is dimensional, not competence: Anthropic identified the right operational pressure and solved the token-management subset. This paper provides the framework that explains both when their approach is sufficient and when uncalibrated compression is destructive.

Memory architectures such as MemGPT [4] and related systems (Mem0, Claude Code’s memory features) address persistence—maintaining state across session boundaries. They solve the storage problem: where does context live between sessions? Gate Calibration addresses the

quality problem: how much meaning survives each compression cycle, and how does the system know? The two concerns are complementary. A persistent system needs both storage and calibration. Current memory architectures provide storage without calibration—they persist context but do not measure what compression loses.

Self-managing memory systems—architectures that autonomously decide what to compress, retain, or discard—face the Gödel constraint directly. The system promoting provisional notes to verified knowledge, consolidating session records without external review, or summarizing its own accumulated context is performing the self-referential operation that the incompleteness result [3] prohibits. Gate Calibration provides the architectural response: external verification at the compression boundary, with the calibration loop generating the data that makes verification systematic rather than ad hoc.

9 Implications

9.1 The Context Architect

The calibration framework implies a professional role: the Context Architect. This is the practitioner who configures and calibrates persistent AI deployments. The configuration surface includes: model selection per role (determining W and baseline D), use-case profiling (determining fidelity profile and F_{target} per dimension), structural protection (identifying content outside the compression cycle), provenance mapping (classifying sources into the three-bin framework), gate calibration (initial conservative gates refined through protocol), and output instrumentation (specifying return-signal structure for the calibration loop).

The system manages token position tracking, zone detection, gate enforcement, and calibration data collection automatically. The Context Architect manages the parameters that shape how these mechanisms behave. This division—human configures, system executes, human calibrates, system adapts—is the operational expression of the Gödel constraint: automated execution within externally calibrated boundaries.

9.2 Enterprise Application

For enterprise deployments, the four-dimensional fidelity framework produces a configuration and monitoring surface—dimensional compression scores, gate-hit frequency, and cost trending per deployment—that makes the calibration state of an organization’s AI systems legible to engineering leadership.

9.3 Forward Pointers

Three lines of subsequent work extend directly from this paper.

Four-Dimensional Fidelity Benchmark. No existing benchmark tests the four fidelity dimensions independently. Current evaluation suites collapse compression quality into aggregate scores that mask dimensional failure. Developing a measurement suite that rates each dimension independently—with inter-rater reliability sufficient for calibration purposes—is the prerequisite for systematic cross-model comparison.

Degradation Fine Structure. This paper treats D as aggregate and introduces provenance as the featured modulator. The full structure of what drives degradation rate includes additional modulators—task-type effects on effective degradation, the interaction between task structure and dimensional profiles, and model-specific dimensional degradation characteristics. These constitute the fine structure of degradation, addressed in subsequent work.

Cross-Cycle Compounding. The two-pass architecture bounds per-cycle degradation but does not resolve whether compound degradation across many cycles accumulates, stabilizes, or exhibits more complex dynamics. Longitudinal measurement across dozens of compression cycles, with dimensional tracking, is needed to characterize the compounding behavior.

10 Conclusion

The treatment for persistent AI memory loss is not a better memory. It is a better instrument for measuring what compression destroys.

The four-dimensional fidelity decomposition provides the diagnostic vocabulary—factual accuracy, structural coherence, trajectory preservation, and tonal/strategic nuance are independent axes along which compression quality can be measured, tracked, and managed. The calibration loop provides the protocol—conservative initialization, external measurement, iterative refinement, steady state. Provenance-aware compression provides the efficiency lever—concentrating fidelity budget on irreplaceable signal rather than recoverable facts. The two-pass compression architecture provides the safety guarantee—the highest-stakes compression always runs in the cleanest channel.

What the framework enables is persistent AI systems that improve rather than degrade—systems where each compression cycle is externally measured, each measurement refines the next gate position, and the compounding effect runs toward precision rather than drift. What remains open is the measurement infrastructure itself: dimensional benchmarks with inter-rater reliability, longitudinal compounding dynamics across dozens of cycles, and the fine structure of how task type and model architecture interact to produce dimensional degradation profiles. These are engineering problems with engineering solutions, and the four-dimensional framework provides the conceptual apparatus for pursuing them.

The independence of fidelity duration from window size reframes the scaling conversation. Larger windows provide more capacity; they do not provide more fidelity. The binding constraint for serious deployments is not how many tokens the system can carry but how long it can think well. The human role in persistent AI systems is not being automated away. It is being elevated—from operator to measurement infrastructure, from bottleneck to the engineering instrument that makes calibrated persistence possible.

References

- [1] P. Navid. Automatic Context Compaction. Anthropic Cookbook, November 2025. <https://platform.claude.com/cookbook/tool-use-automatic-context-compaction>

- [2] B. Odriozola-Schick. The Root Theorem of Context Engineering. *arXiv preprint*, arXiv:7419948, 2026.
- [3] B. Odriozola-Schick. On the Incompleteness of Self-Compressing Systems. *arXiv preprint*, arXiv:7442484, 2026.
- [4] C. Packer, S. Wooders, K. Lin, V. Fang, S. G. Patil, and J. E. Gonzalez. MemGPT: Towards LLMs as Operating Systems. *arXiv:2310.08560*, 2023.